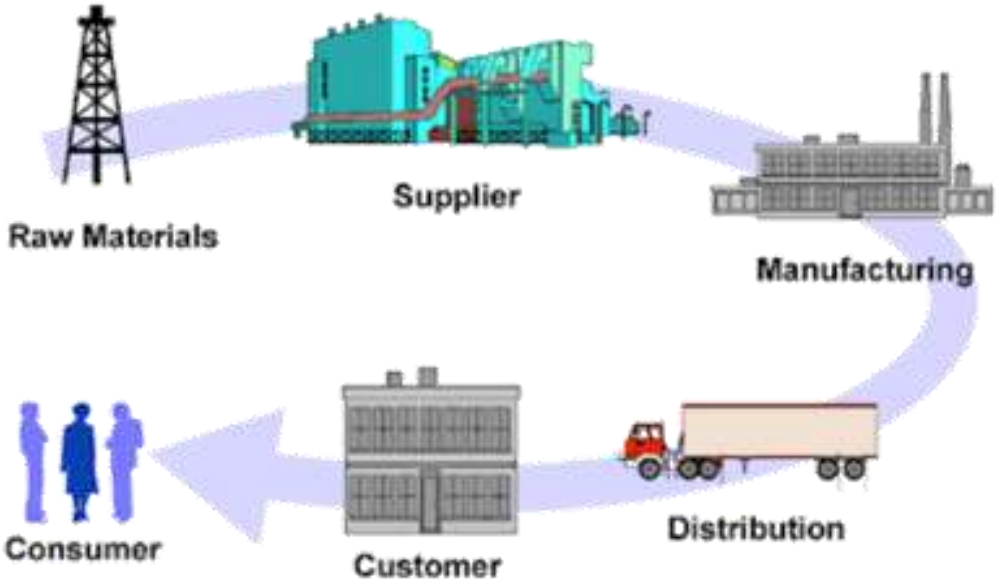
Unit 1

Manufacturing definition

Manufacturing definition there are two ways

1. manufacturing is the process of converting a raw material component or a part into a finished good that meets a customer expectation or specification, that is manufacturing,
2. manufacturing is a value adding process, allowing businesses to sell finished products at a premium over the value of the raw materials used.



CAD/CAM

- Computer-aided design (CAD) is the use of computer systems to assist in the creation, modification, analysis, or optimization of a design.

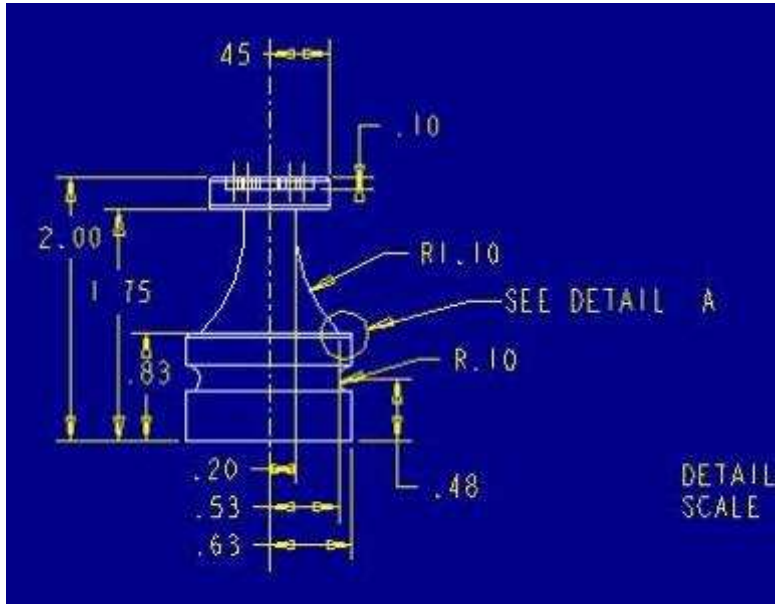
- Computer-aided manufacturing (CAM) is the use of computer systems to plan, manage, and control the operations of a manufacturing plant through direct or indirect computer interface with plant's resources.

Reasons for implementing CAD

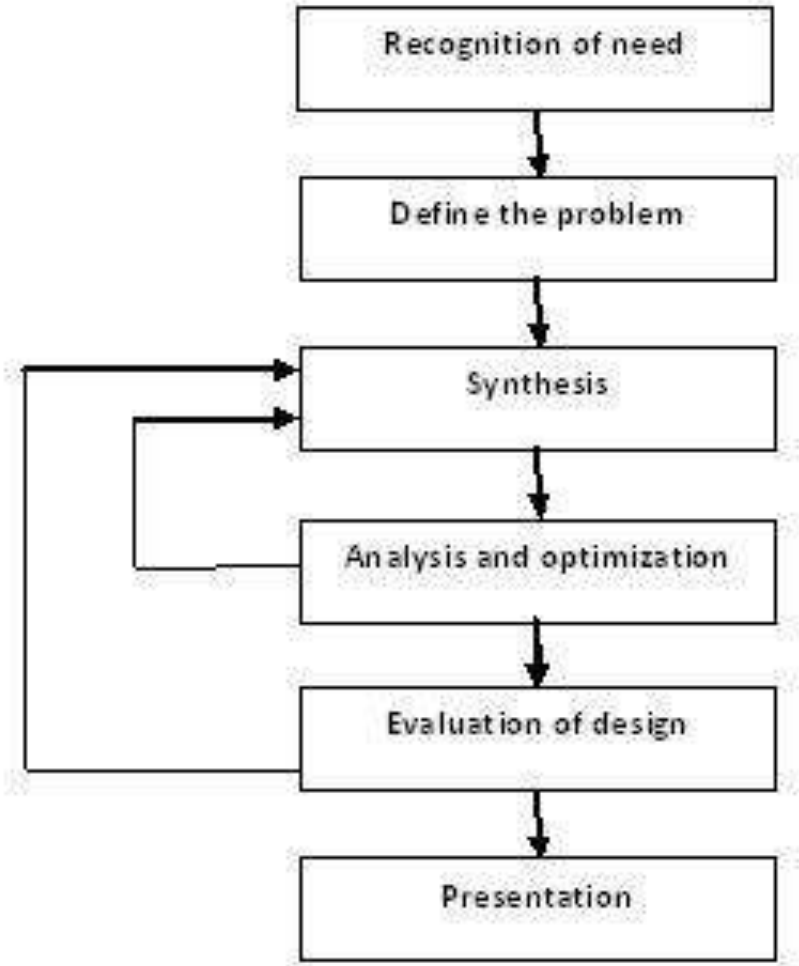
- To increase productivity of the designer
- To improve quality of the design
- To improve communications
- To create a manufacturing database
- To create and test toolpaths and optimize them
- To help in production scheduling and MRP models
- To have effective shop floor control

How do CAD/CAM systems work?

- Developing NC code requires an understanding of:
  1. Part geometry
  2. Tooling
  3. Process plans
  4. Tolerances
  5. Fixturing
- Most CAD/CAM systems provide access to:
  1. Part geometry
  2. Tooling



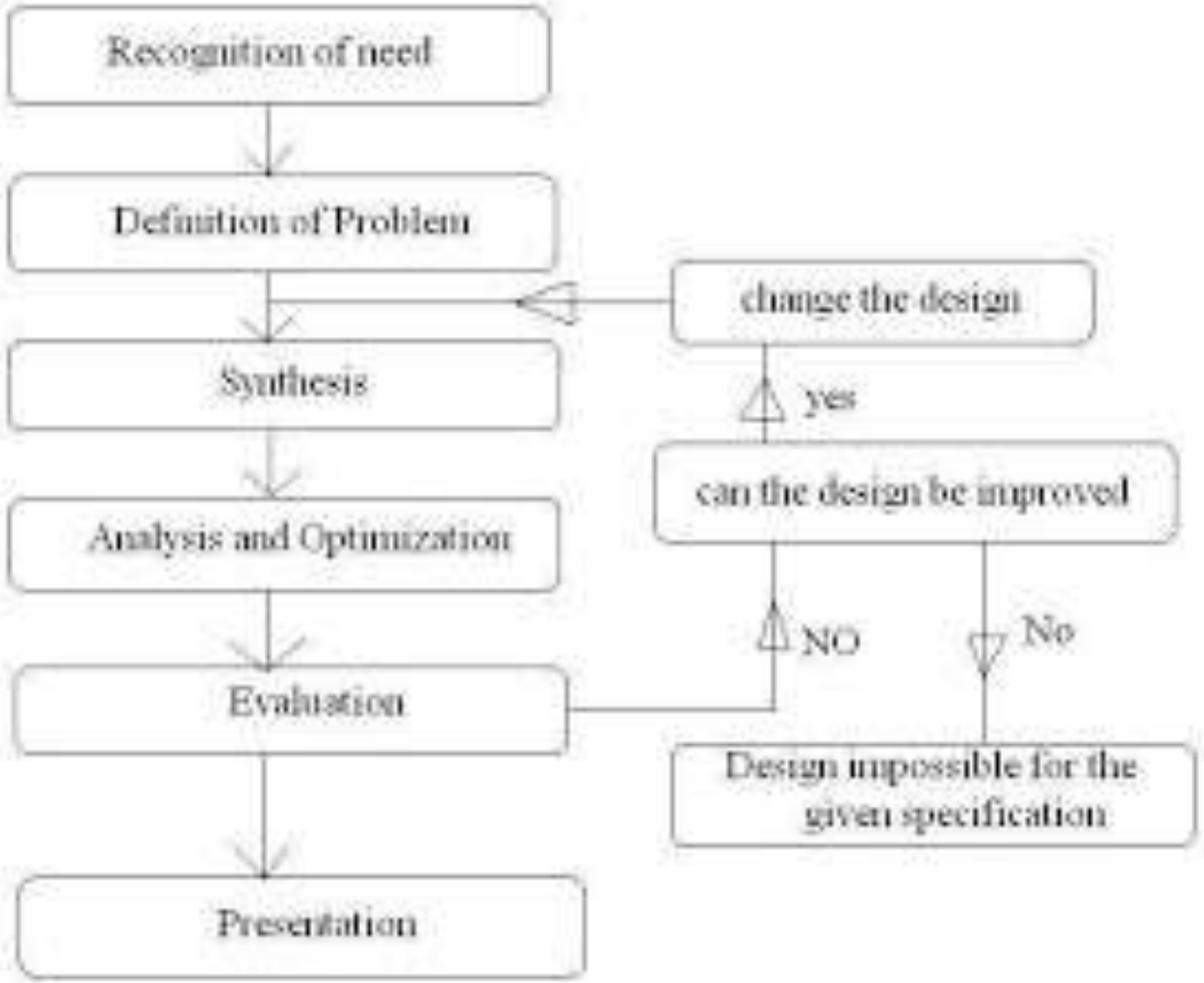
The Design Process



The Design Process

- **Recognition of need**: Occurs when one recognizes the need of a solution to a problem or situation.
- **Problem definition**: This step defines the problem to determine what features a product will contain. Customer surveys also are taken. Characteristics, cost, quality are all defined in this step.
- **Synthesis**: Research is done and preliminary ideas are given. If the same product is available in the market, then unique and better approaches may be taken into consideration.
- **Analysis and optimization**: The product gets analyzed constantly to identify opportunities for optimization. Customer feedback is also solicited so that production errors can be reduced and improved.
- **Evaluation**: This step involves design testing through a prototype of a product. The quality and reliability of the product is also evaluated.
- **Presentation**: This step shows the final product drawings and specifications and involves presentation of modules, 2D diagrams, and 3D diagrams.

The Design Process (Shigley Model)

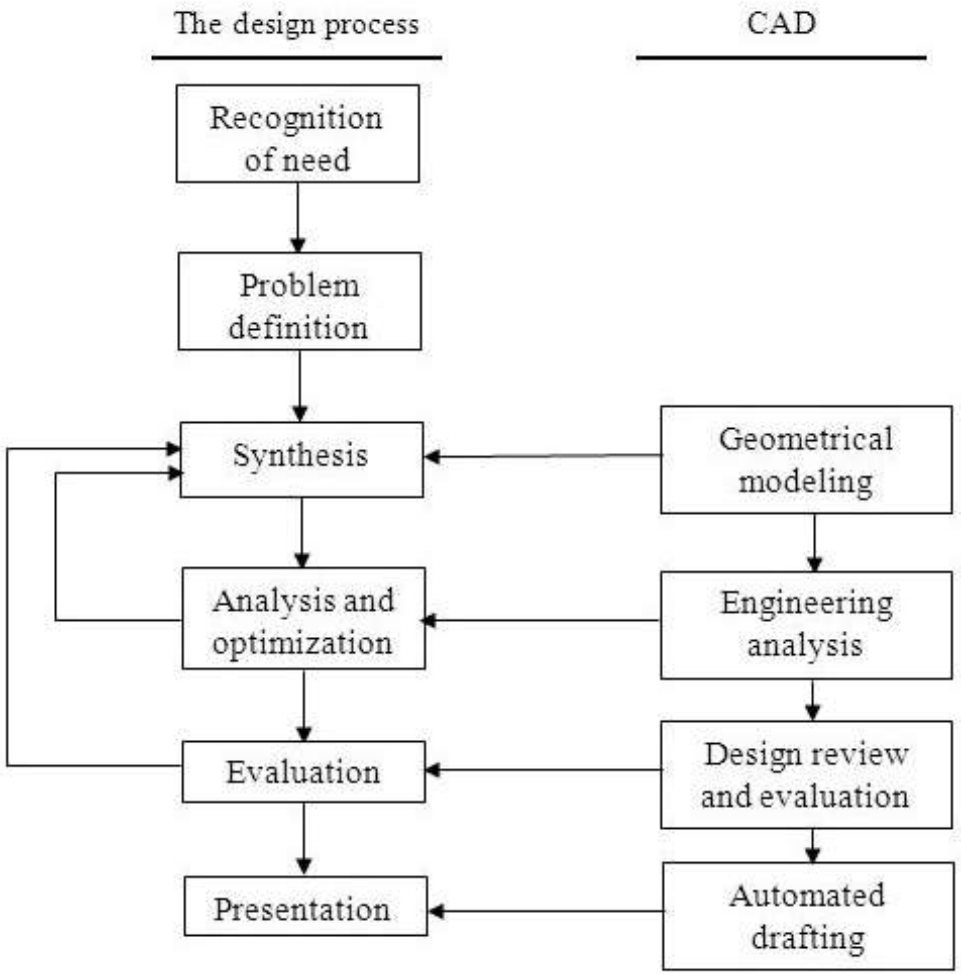


The Design Process (Shigley Model)

There are four steps in the CAD design process:

- **Geometric modeling** (in place of synthetic step): This is a mathematical technique that is done by using simulation software. It shows the object from the math formula through computer graphics.
- Engineering analysis **(in place of analysis and optimization): This is the process of checking the strength of moving objects to determine how they will work in the real world.**
- **Design review and evaluation** (replaces evaluation): Design is reviewed and evaluated with the help of CAD software. Also, the feasibility of the design is checked.
- **Automatic drafting** (replaces presentation): The design is checked to see if it meets product specifications. The design is then saved automatically.

Application of computers in design



Benefits of CAD

The CAD systems are user friendly. The work can be completed very quickly. This results in reduction of labour and time.

- The computers work very accurately, giving better quality drawing.
- The drawings can be stored and recollected when ever needed.
- The corrections and modifications can be done quickly and easily.
- Different design ideas can be tried having single basic drawing.
- Visual modelling of any object or component can be done.
- When the same component is needed in many places, the same drawing can be used.
- It helps to visualize how the final product will be by using colour graphics.

Introduction of Computer Graphics

- Computer Graphics involves technology to access. The Process transforms and presents information in a visual form. The role of computer graphics insensible. In today life, computer graphics has now become a common element in user interfaces, T.V. commercial motion pictures.
- Computer Graphics is the creation of pictures with the help of a computer. The end product of the computer graphics is a picture it may be a business graph, drawing, and engineering.
- In computer graphics, two or three-dimensional pictures can be created that are used for research. Many hardware devices algorithm has been developing for improving the speed of picture generation with the passes of time. It includes the creation storage of models and image of objects. These models for various fields like engineering, mathematical and so on.
- Today computer graphics is entirely different from the earlier one. It is not possible. It is an interactive user can control the structure of an object of various input devices.

Types of Computer graphics

1. Non Interactive graphics : where user does not have any kind of control over the image
2. Interactive graphics : It involves a two way communication between computer and user

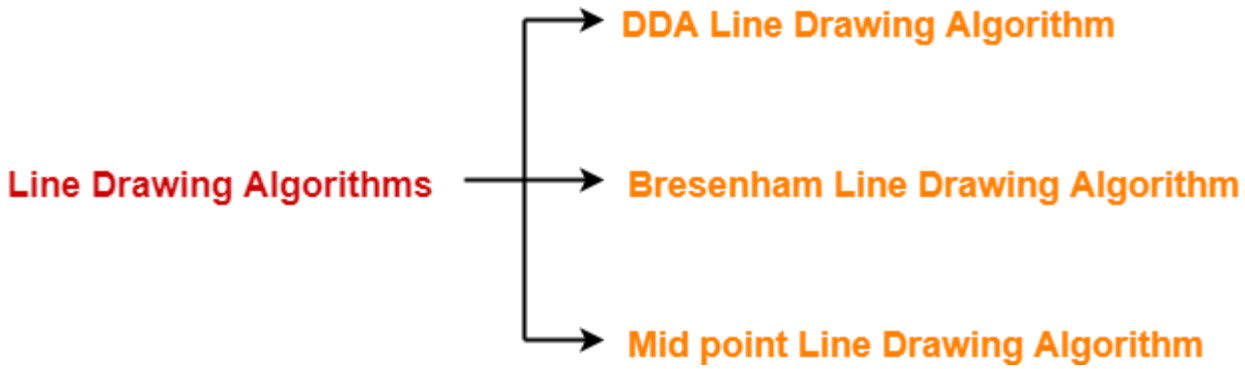
Points and lines

- Point is the fundamental element of picture representation
- It is the position in the plan defined as either pair or triplets of number depending upon the dimension
- Two points represent line or edge and 3 or more points a polygon
- Curved lines are represented by the short straight lines

Principle of Interactive computer Graphics

Line Drawing Algorithms

In computer graphics, popular algorithms used to generate lines are-



1. Digital Differential Analyzer (DDA) Line Drawing Algorithm
2. Bresenham Line Drawing Algorithm
3. Mid Point Line Drawing Algorithm

Line Drawing Algorithms

The process of tuning on the pixels foe a line segment is called vector generation or line generation, and the algorithms foe tem are known as vector generation algorithms or line drawing algorithms,

Before discussing the specific line drawing algorithms it is useful to note the general requirements for such algorithms. these requirements specify the desired characteristics of line

- The line should appear as a straight line and it should start and end accurately.
- The line should be displayed with constant brightness along its length independent of its length and orientation
- The line should be drawn rapidly.

Digital Differential Analyser(DDA) Algorithm

- It is a scan conversion method for drawing lines on screen
- It uses incremental approach
- Difference in pixel point is to be analysed

Line Drawing Algorithms

- Slope-Intercept Equation

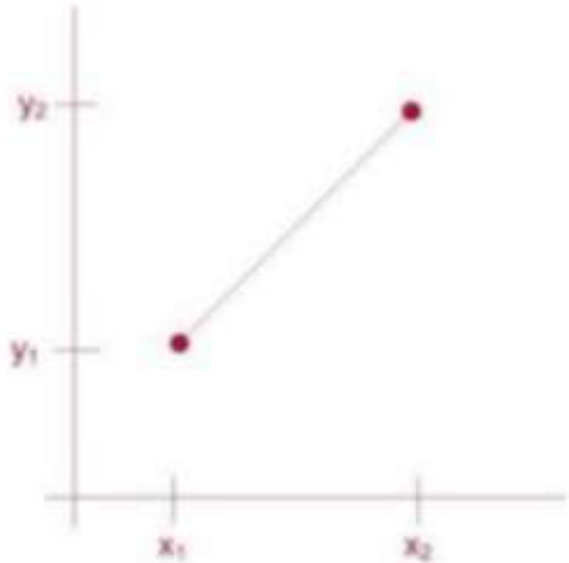
$$y = m.x + b$$

- Slope

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

- Intercept

$$b = y_1 - m.x_1$$



- Interval Calculation

$$\Delta y = m.\Delta x \qquad \Delta x = \frac{\Delta y}{m}$$

Digital Differential Analyser(DDA) Algorithm

Procedure-

Starting coordinates = (X_0, Y_0)

Ending coordinates = (X_n, Y_n)

The points generation using DDA Algorithm involves the following steps-

- **Step-01:**

Calculate ΔX, ΔY and M from the given input.

These parameters are calculated as-

$\Delta X = X_n - X_0$

$\Delta Y = Y_n - Y_0$

$M = \Delta Y / \Delta X$

Digital Differential Analyser(DDA) Algorithm

Step-02:

- Find the number of steps or points in between the starting and ending coordinates.

if (absolute (ΔX) > absolute (ΔY))

Steps = absolute (ΔX);

else

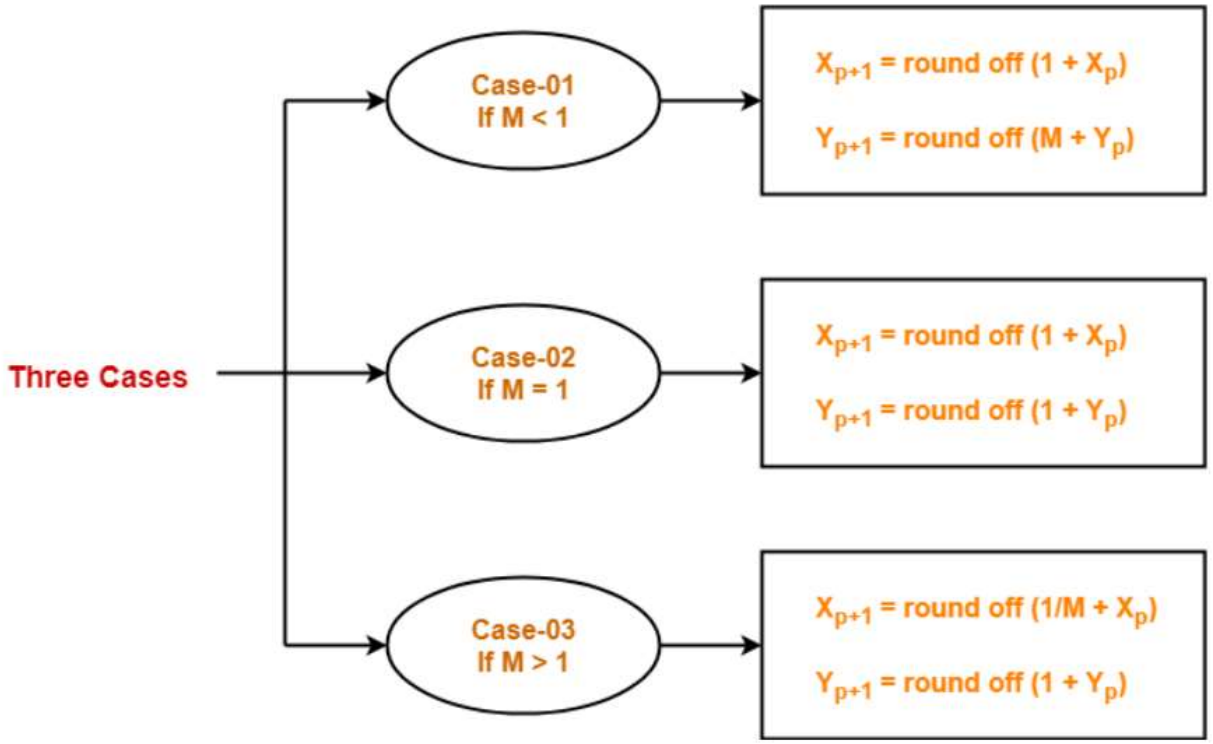
Steps = absolute (ΔY);

Digital Differential Analyser(DDA) Algorithm

- **Step-03:**

i. Suppose the current point is (X_p, Y_p) and the next point is (X_{p+1}, Y_{p+1}).

ii. Find the next point by following the below three cases-



Digital Differential Analyser(DDA) Algorithm

- **Step-04:**

Keep repeating Step-03 until the end point is reached or the number of generated new points (including the starting and ending points) equals to the steps count.

Digital Differential Analyser(DDA) Algorithm

- **Problem-01:**

Calculate the points between the starting point (5, 6) and ending point (8, 12).

- **Solution-**

- Starting coordinates = $(X_0, Y_0) = (5, 6)$
- Ending coordinates = $(X_n, Y_n) = (8, 12)$

- **Step-01:**

- Calculate ΔX, ΔY and M from the given input.

$\Delta X = X_n - X_0 = 8 - 5 = 3$

$\Delta Y = Y_n - Y_0 = 12 - 6 = 6$

$M = \Delta Y / \Delta X = 6 / 3 = 2$

Digital Differential Analyser(DDA) Algorithm

- **Step-02:**

Calculate the number of steps.

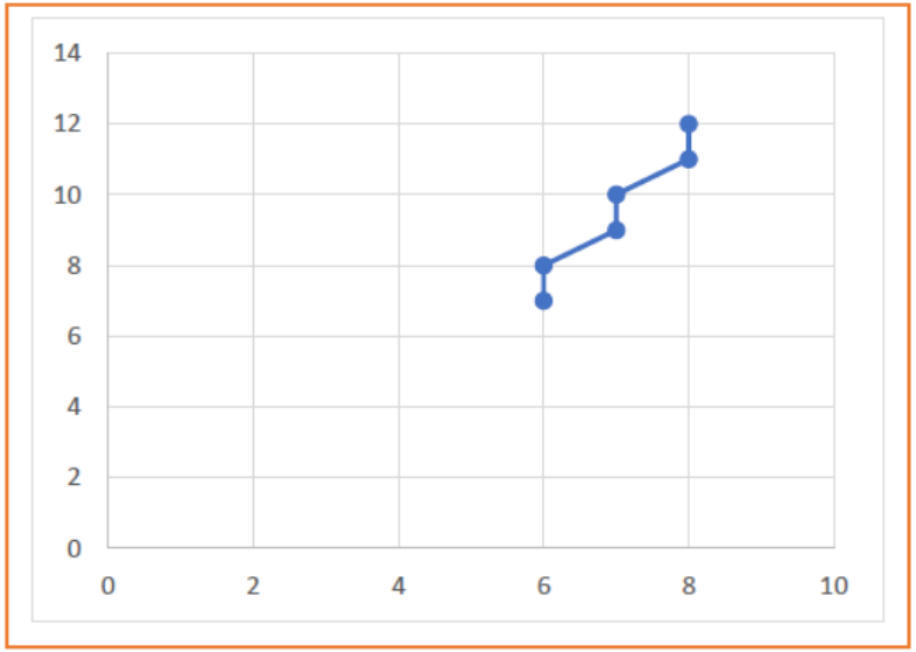
As $|\Delta X| < |\Delta Y| = 3 < 6$, so number of steps = $\Delta Y = 6$

- **Step-03:**

- As M > 1, so case-03 is satisfied.
- Now, Step-03 is executed until Step-04 is satisfied.

Digital Differential Analyser(DDA) Algorithm

X_p	Y_p	X_{p+1}	Y_{p+1}	Round off (X_{p+1}, Y_{p+1})
5	6	5.5	7	(6, 7)
		6	8	(6, 8)
		6.5	9	(7, 9)
		7	10	(7, 10)
		7.5	11	(8, 11)
		8	12	(8, 12)



Bresenham Line Drawing Algorithm

- **Procedure-**

Given

Starting coordinates = (X_0, Y_0)

Ending coordinates = (X_n, Y_n)

The points generation using Bresenham Line Drawing Algorithm involves the following steps-

- **Step-01:**

Calculate ΔX and ΔY from the given input.

These parameters are calculated as-

$\Delta X = X_n - X_0$

$\Delta Y = Y_n - Y_0$

- **Step-02:**

Calculate the decision parameter P_k.

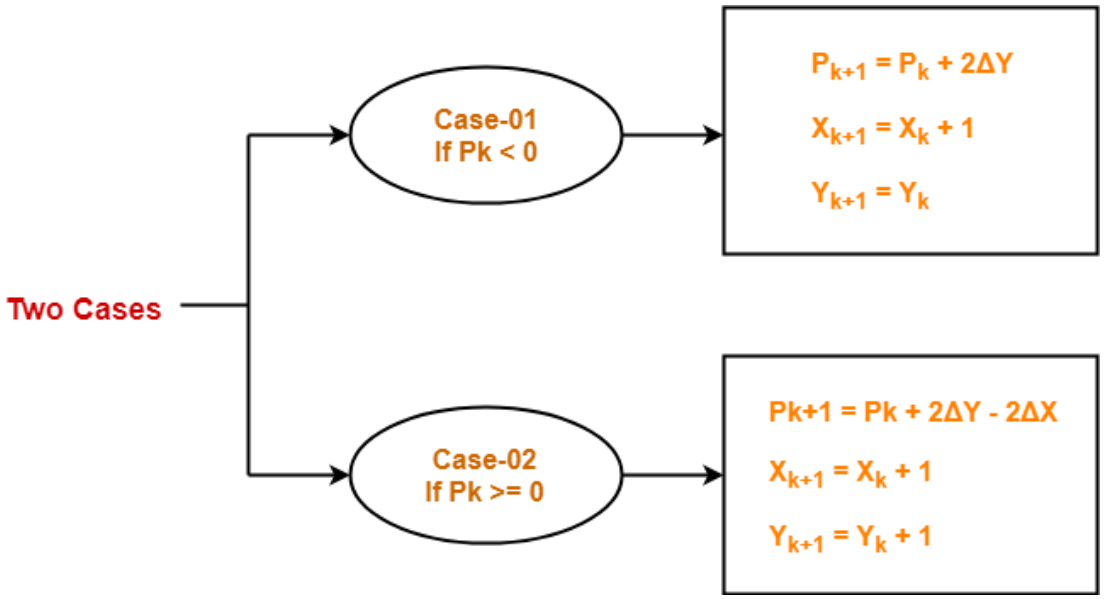
It is calculated as-

$P_k = 2\Delta Y - \Delta X$

Bresenham Line Drawing Algorithm

Step-03:

- Suppose the current point is (X_k, Y_k) and the next point is (X_{k+1}, Y_{k+1}).
- Find the next point depending on the value of decision parameter P_k.
- Follow the below two cases-



- **Step-04:**

Keep repeating Step-03 until the end point is reached or number of iterations equals to (ΔX-1) times.

Problem-01:

- Calculate the points between the starting coordinates (9, 18) and ending coordinates (14, 22).

Solution-

Starting coordinates = $(X_0, Y_0) = (9, 18)$

Ending coordinates = $(X_n, Y_n) = (14, 22)$

- **Step-01:**

Calculate ΔX and ΔY from the given input.

$\Delta X = X_n - X_0 = 14 - 9 = 5$

$\Delta Y = Y_n - Y_0 = 22 - 18 = 4$

- **Step-02:**

Calculate the decision parameter.

$P_k = 2\Delta Y - \Delta X$

$= 2 \times 4 - 5$

$= 3$

So, decision parameter $P_k = 3$

- **Step-03:**

As $P_k >= 0$, so case-02 is satisfied.

Thus,

$P_{k+1} = P_k + 2\Delta Y – 2\Delta X = 3 + (2 \times 4) – (2 \times 5) = 1$

$X_{k+1} = X_k + 1 = 9 + 1 = 10$

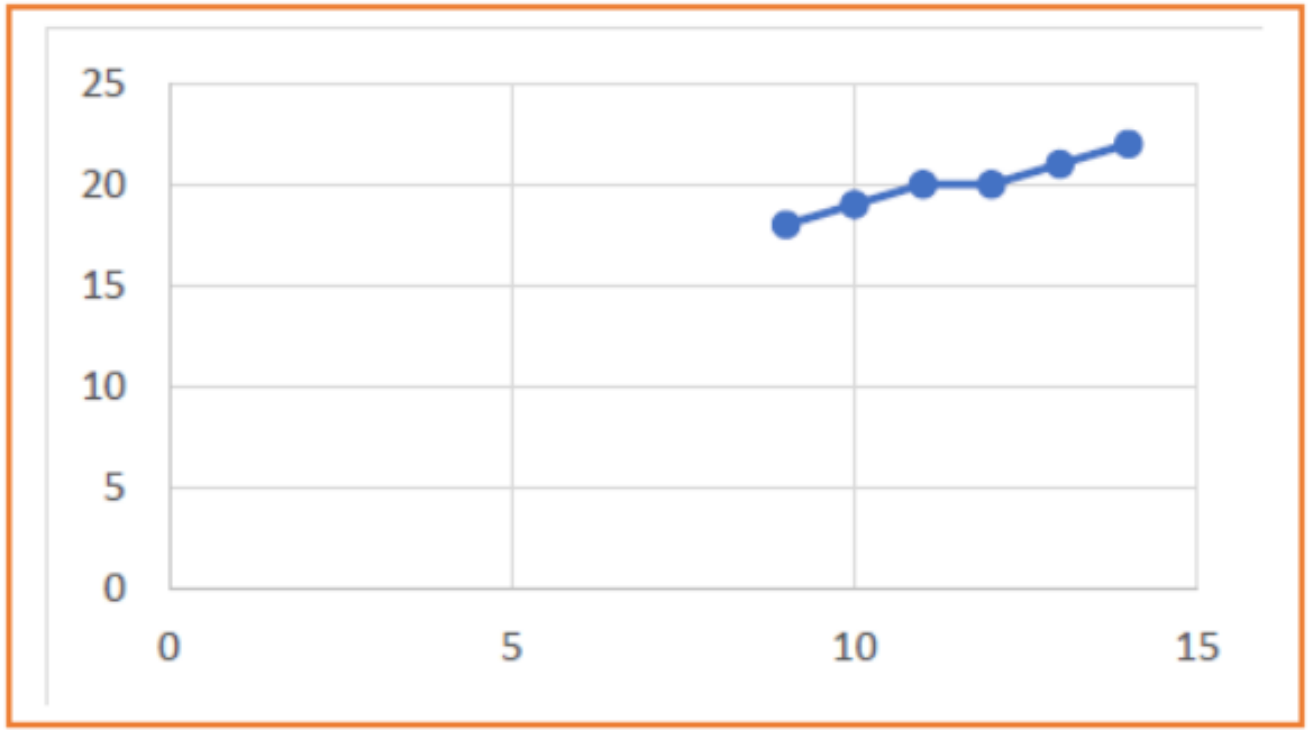
$Y_{k+1} = Y_k + 1 = 18 + 1 = 19$

Similarly, Step-03 is executed until the end point is reached or number of iterations equals to 4 times.

(Number of iterations = $\Delta X – 1 = 5 – 1 = 4$)

Bresenham Line Drawing Algorithm

P_k	P_{k+1}	X_{k+1}	Y_{k+1}
		9	18
3	1	10	19
1	-1	11	20
-1	7	12	20
7	5	13	21
5	3	14	22



Difference between DDA and Bresenham's Algorithm

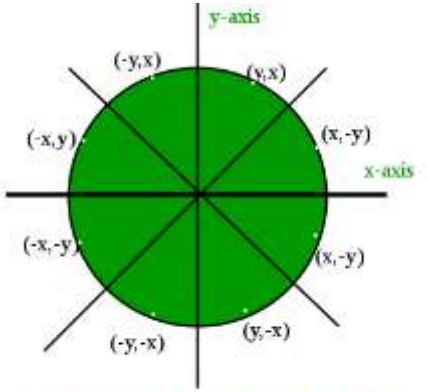
DDA Algorithm	Bresenham's Line Algorithm
1. DDA Algorithm use floating point, i.e., Real Arithmetic.	1. Bresenham's Line Algorithm use fixed point, i.e., Integer Arithmetic
2. DDA Algorithms uses multiplication & division its operation	2.Bresenham's Line Algorithm uses only subtraction and addition its operation
3. DDA Algorithm is slowly than Bresenham's Line Algorithm in line drawing because it uses real arithmetic (Floating Point operation)	3. Bresenham's Algorithm is faster than DDA Algorithm in line because it involves only addition & subtraction in its calculation and uses only integer arithmetic.
4. DDA Algorithm is not accurate and efficient as Bresenham's Line Algorithm.	4. Bresenham's Line Algorithm is more accurate and efficient at DDA Algorithm.
5.DDA Algorithm can draw circle and curves but are not accurate as Bresenham's Line Algorithm	5. Bresenham's Line Algorithm can draw circle and curves with more accurate than DDA Algorithm.

Bresenham’s circle drawing algorithm

- It is not easy to display a continuous smooth arc on the computer screen as our computer screen is made of pixels organized in matrix form. So, to draw a circle on a computer screen we should always choose the nearest pixels from a printed pixel so as they could form an arc. There are two algorithm to do this:
- Mid-Point circle drawing algorithm
- Bresenham’s circle drawing algorithm

Bresenham’s circle drawing algorithm

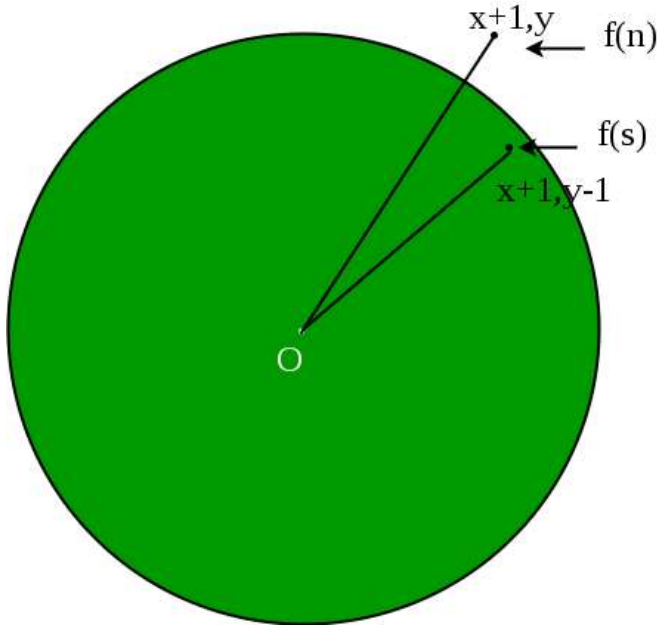
- Both of these algorithms uses the key feature of circle that it is highly symmetric. So, for whole 360 degree of circle we will divide it in 8-parts each octant of 45 degree. In order to that we will use Bresenham’s Circle Algorithm for calculation of the locations of the pixels in the first octant of 45 degrees. It assumes that the circle is centered on the origin. So for every pixel (x, y) it calculates, we draw a pixel in each of the 8 octants of the circle as shown below :



For a pixel (x,y) all possible pixels in 8 octants.

Bresenham's circle drawing algorithm

- Now, we will see how to calculate the next pixel location from a previously known pixel location (x, y). In Bresenham's algorithm at any point (x, y) we have two option either to choose the next pixel in the east i.e. (x+1, y) or in the south east i.e. (x+1, y-1).



Bresenham's circle drawing algorithm

- And this can be decided by using the decision parameter d as:
  If d > 0, then (x+1, y-1) is to be chosen as the next pixel as it will be closer to the arc.
- else (x+1, y) is to be chosen as next pixel.
- Now to draw the circle for a given radius 'r' and centre (xc, yc) We will start from (0, r) and move in first quadrant till x=y (i.e. 45 degree). We should start from listed initial condition:

```
d = 3 - (2 * r)
x = 0
y = r
```

Bresenham's circle drawing algorithm

Now for each pixel, we will do the following operations:

- Set initial values of (xc, yc) and (x, y)
- Set decision parameter d to d = 3 – (2 * r).

- call drawCircle(int xc, int yc, int x, int y) function.
- Repeat steps 5 to 8 until x < = y
- Increment value of x.
- If d < 0, set d = d + (4*x) + 6
- Else, set d = d + 4 * (x – y) + 10 and decrement y by 1.

Bresenham's circle drawing algorithm

- **Advantages**
- It is a simple algorithm.
- It can be implemented easily
- It is totally based on the equation of circle i.e. $x^2 + y^2 = r^2$
- **Disadvantages**
- There is a problem of accuracy while generating points.
- This algorithm is not suitable for complex and high graphic images.

Scan Conversion

- Definition: It is a process of representing graphics objects a collection of pixels. The graphics objects are continuous. The pixels used are discrete. Each pixel can have either on or off state.
- Primitives are defined using points, which have been mapped to the screen coordinates.
  i. In vector graphics, connect the points using a pen directly.
  ii. In Raster Graphics, we create a discretized image of the whole screen onto the frame buffer first. The image is scanned automatically onto the display periodically.
  iii. This step is called Scan Conversion or Rasterization

Scan Conversion

Advantage of developing algorithms for scan conversion

- Algorithms can generate graphics objects at a faster rate.
- Using algorithms memory can be used efficiently.
- Algorithms can develop a higher level of graphical objects.

Examples of objects which can be scan converted

- Point
- Line
- Sector
- Arc
- Ellipse
- Rectangle
- Polygon
- Characters
- Filled Regions

Converting a vector into a raster is called rasterization. The algorithms implementation varies from one computer system to another computer system. Some algorithms are implemented using the software. Some are performed using hardware or firmware. Some are performed using various combinations of hardware, firmware, and software.

Introduction of Transformations

- Computer Graphics provide the facility of viewing object from different angles. The architect can study building from different angles i.e.
  i. Front Elevation
  ii. Side elevation
  iii. Top plan
- A Cartographer can change the size of charts and topographical maps. So if graphics images are coded as numbers, the numbers can be stored in memory. These numbers are modified by mathematical operations called as Transformation.
- The purpose of using computers for drawing is to provide facility to user to view the object from different angles, enlarging or reducing the scale or shape of object called as Transformation.

Transformations

- Two essential aspects of transformation are given below:
- Each transformation is a single entity. It can be denoted by a unique name or symbol.
- It is possible to combine two transformations, after connecting a single transformation is obtained, e.g., A is a transformation for translation. The B transformation performs scaling. The combination of two is C=AB. So C is obtained by concatenation property.

There are two complementary points of view for describing object transformation.

- Geometric Transformation: The object itself is transformed relative to the coordinate system or background. The mathematical statement of this viewpoint is defined by geometric transformations applied to each point of the object.
- Coordinate Transformation: The object is held stationary while the coordinate system is transformed relative to the object. This effect is attained through the application of coordinate transformations.

Transformations

An example that helps to distinguish these two viewpoints:

The movement of an automobile against a scenic background we can simulate this by

- Moving the automobile while keeping the background fixed- (Geometric Transformation)
- We can keep the car fixed while moving the background scenery- (Coordinate Transformation)

Types of Transformations

- Translation
- Scaling
- Rotating
- Reflection
- Shearing

Scaling

- It is used to alter or change the size of objects. The change is done using scaling factors. There are two scaling factors, i.e. S_x in x direction S_y in y-direction. If the original position is x and y. Scaling factors are S_x and S_y then the value of coordinates after scaling will be x^1 and y^1.
- If the picture to be enlarged to twice its original size then $S_x = S_y = 2$.
- If $S_x$and S_y are not equal then scaling will occur but it will elongate or distort the picture.
- If scaling factors are less than one, then the size of the object will be reduced.
- If scaling factors are higher than one, then the size of the object will be enlarged.

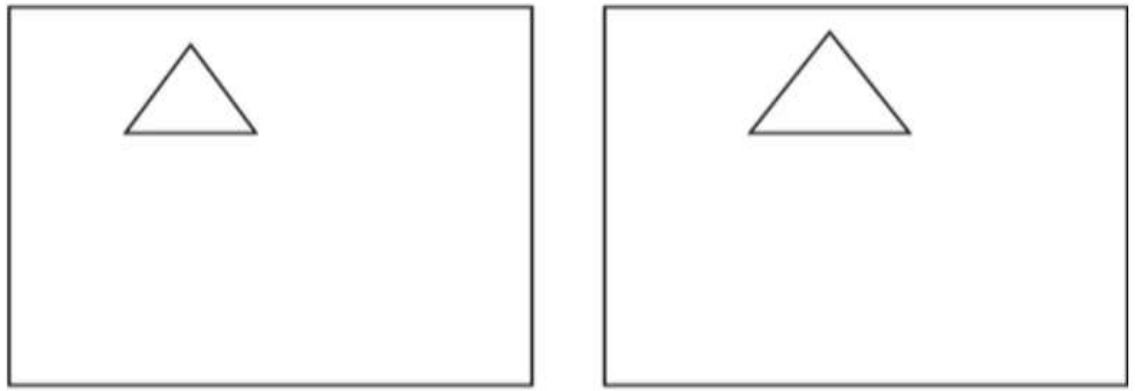
Scaling

- If S_x and S_y are equal it is also called as Uniform Scaling. If not equal then called as Differential Scaling. If scaling factors with values less than one will move the object closer to coordinate origin, while a value higher than one will move coordinate position farther from origin.

Enlargement: If $T_1 = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$, If $(x_1\ y_1)$ is original position and T_1 is translation vector then $(x_2\ y_2)$ are coordinated after scaling

$$[x_2 y_2] = [\,x_1 y_1] \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} = [2x_1\ 2y_1]$$

The image will be enlarged two times



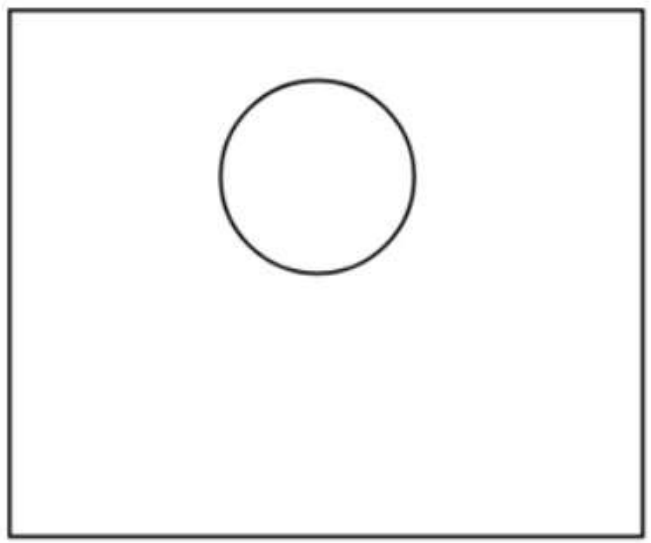
Original Image

Enlarged Image

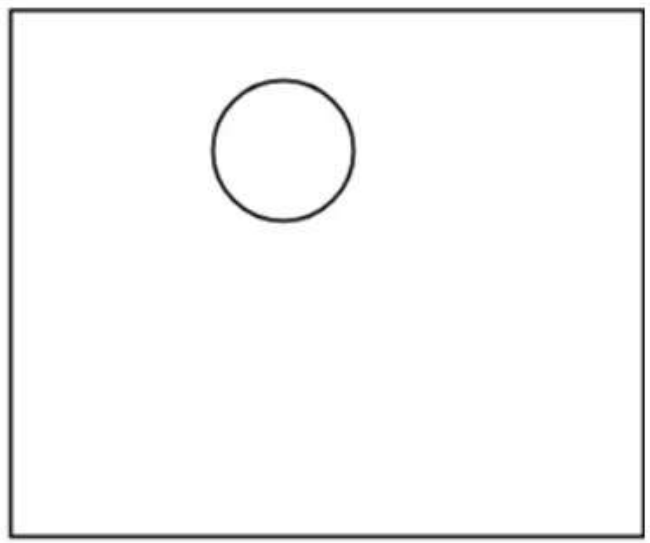
Scaling

Reduction: If $T_1=\begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$. If $(x_1\ y_1)$ is original position and T_1 is translation vector, then $(x_2\ y_2)$ are coordinates after scaling

$$[x_2 y_2] = [\ x_1 y_1] \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} = [.5x_1 . 5y_1]$$

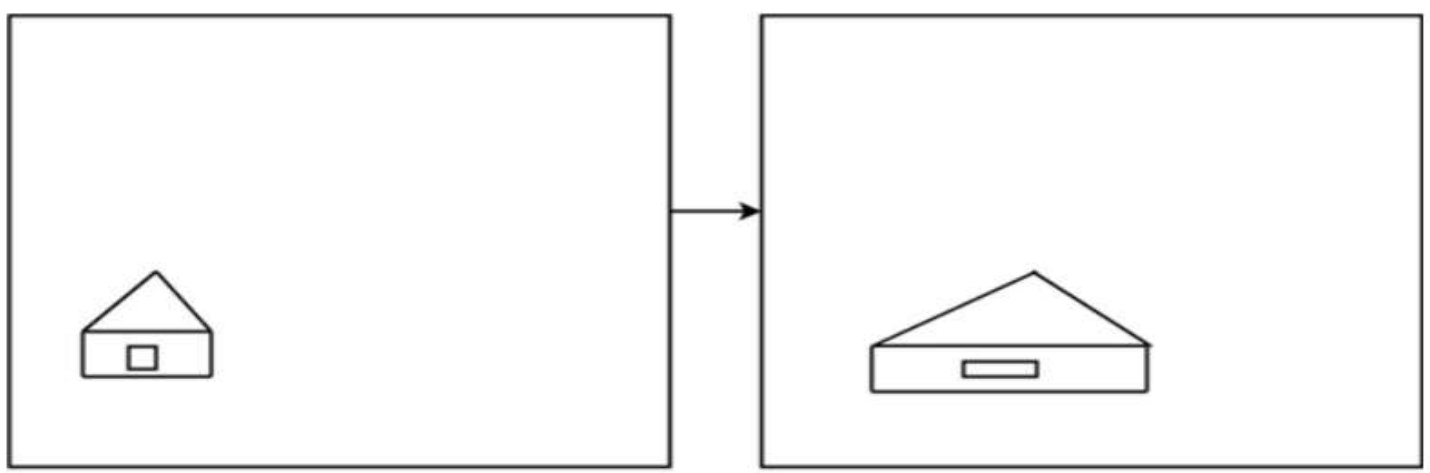


Original Image



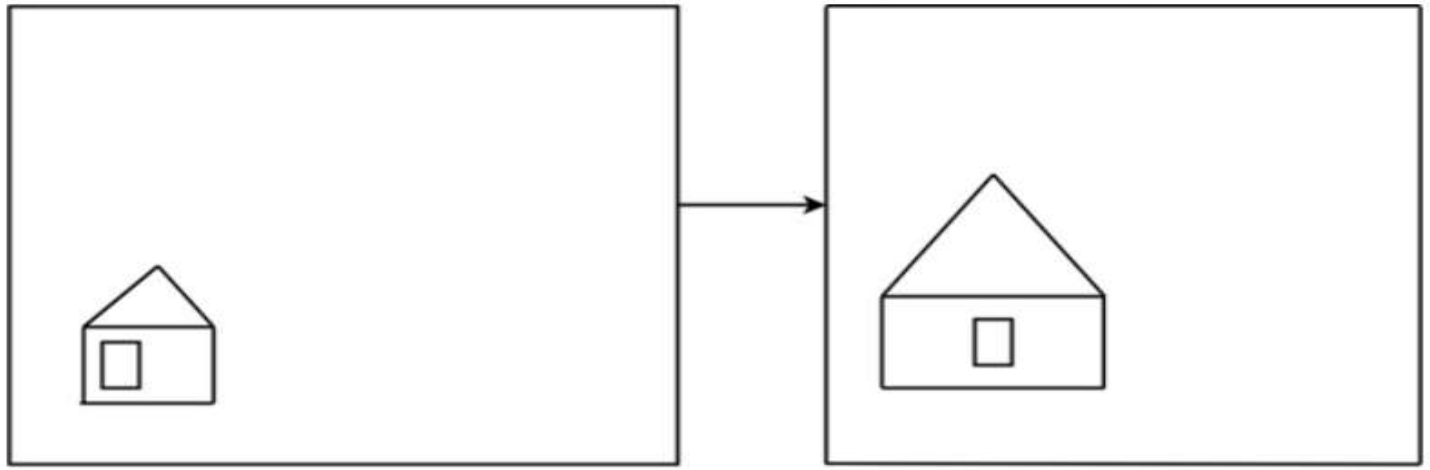
Reduction Image

Scaling



Original Object

Object after scaling in X direction



Original Object

Object after scaling in Y direction

Rotation

Definition : It is a process of changing the angle of the object. Rotation can be clockwise or anticlockwise. For rotation, we have to specify the angle of rotation and rotation point. Rotation point is also called a pivot point. It is print about which object is rotated.

Types of Rotation:

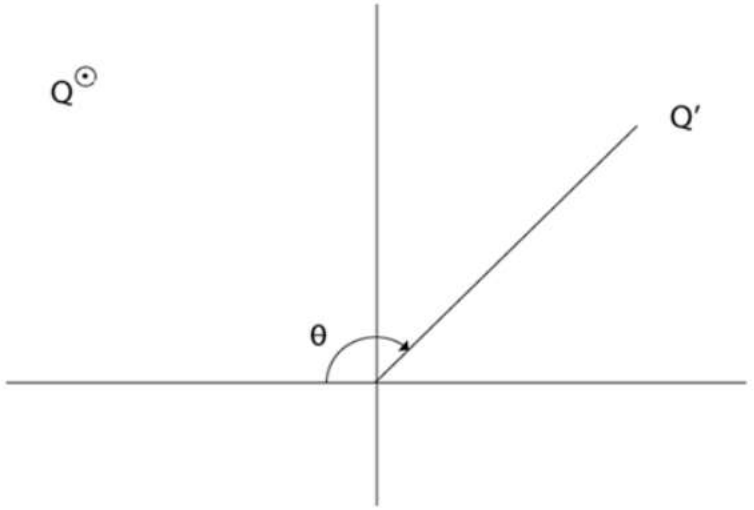
- Anticlockwise
- clockwise
- The positive value of the pivot point (rotation angle) rotates an object in a counter-clockwise (anti-clockwise) direction.
- The negative value of the pivot point (rotation angle) rotates an object in a clockwise direction.
- When the object is rotated, then every point of the object is rotated by the same angle.

Rotation

- **Straight Line:** Straight Line is rotated by the endpoints with the same angle and redrawing the line between new endpoints.
- **Polygon:** Polygon is rotated by shifting every vertex using the same rotational angle.
- **Curved Lines:** Curved Lines are rotated by repositioning of all points and drawing of the curve at new positions.
- **Circle:** It can be obtained by center position by the specified angle.
- **Ellipse:** Its rotation can be obtained by rotating major and minor axis of an ellipse by the desired angle.

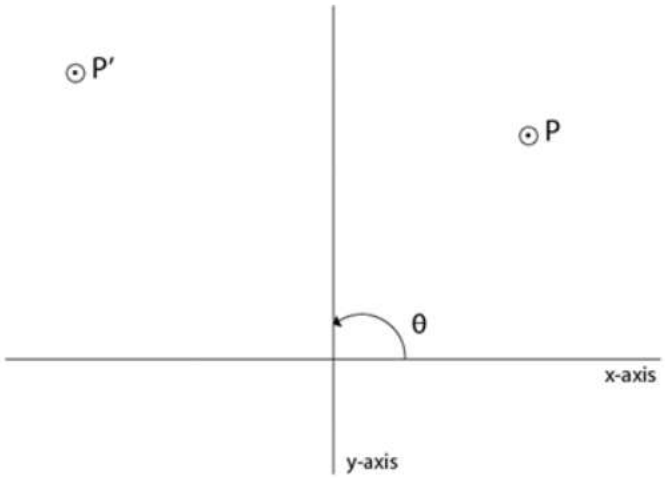
Rotation

Rotation in anticlockwise direction

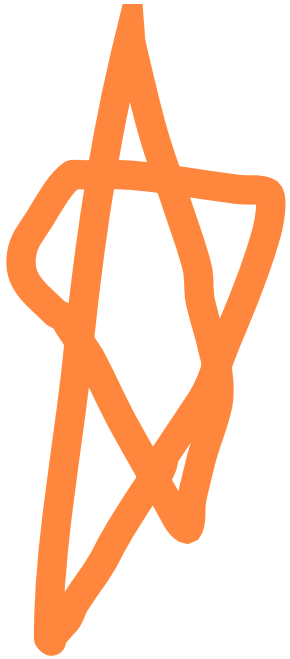


Q is original position
Q ' is final rotated position

Rotation of P in clockwise direction



P is original Position
P' is final position or position after rotation
where θ is angle of rotation



Rotation

Matrix for rotation is a clockwise direction.

$$R=\begin{bmatrix}\cos\theta & \sin\theta \\ \sin\theta & \cos\theta\end{bmatrix}$$

Matrix for rotation is an anticlockwise direction.

$$R=\begin{bmatrix}\cos\theta & \sin\theta \\ -\sin\theta & \cos\theta\end{bmatrix}$$

Matrix for homogeneous co-ordinate rotation (clockwise)

$$R=\begin{pmatrix}\cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1\end{pmatrix}$$

Matrix for homogeneous co-ordinate rotation (anticlockwise)

$$R=\begin{pmatrix}\cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1\end{pmatrix}$$



Rotation about an arbitrary point: If we want to rotate an object or point about an arbitrary point, first of all, we translate the point about which we want to rotate to the origin. Then rotate point or object about the origin, and at the end, we again translate it to the original place. We get rotation about an arbitrary point.

Example: The point (x, y) is to be rotated

Reflection

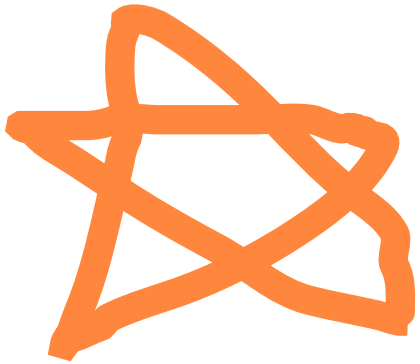
It is a transformation which produces a mirror image of an object. The mirror image can be either about x-axis or y-axis. The object is rotated by 180°.

Types of Reflection:

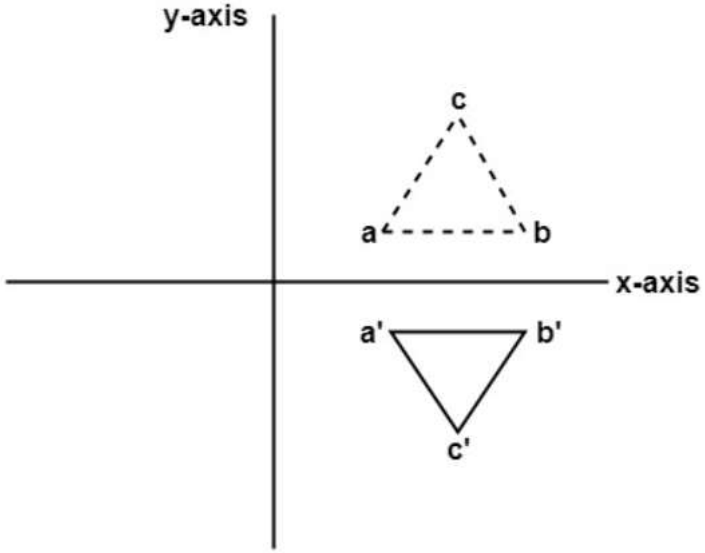
- Reflection about the x-axis
- Reflection about the y-axis
- Reflection about an axis perpendicular to xy plane and passing through the origin
- Reflection about line y=x

Reflection

1. Reflection about x-axis: The object can be reflected about x-axis with the help of the following matrix



$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

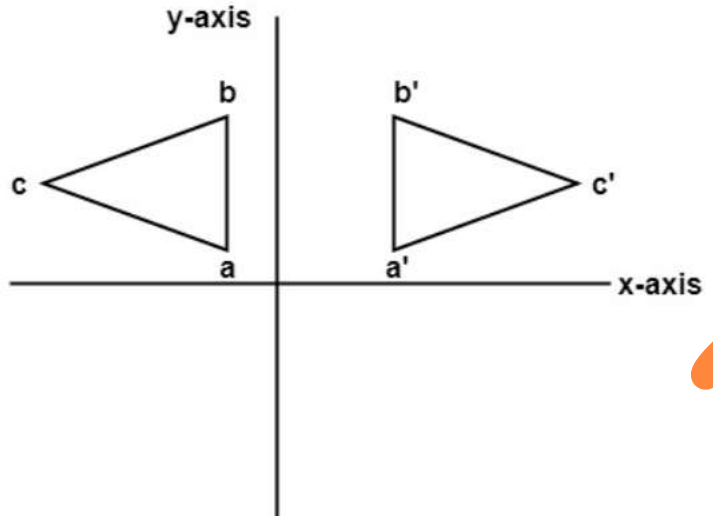


- In this transformation value of x will remain same whereas the value of y will become negative. Following figures shows the reflection of the object axis. The object will lie another side of the x-axis.

Reflection

2. Reflection about y-axis: The object can be reflected about y-axis with the help of following transformation matrix

$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



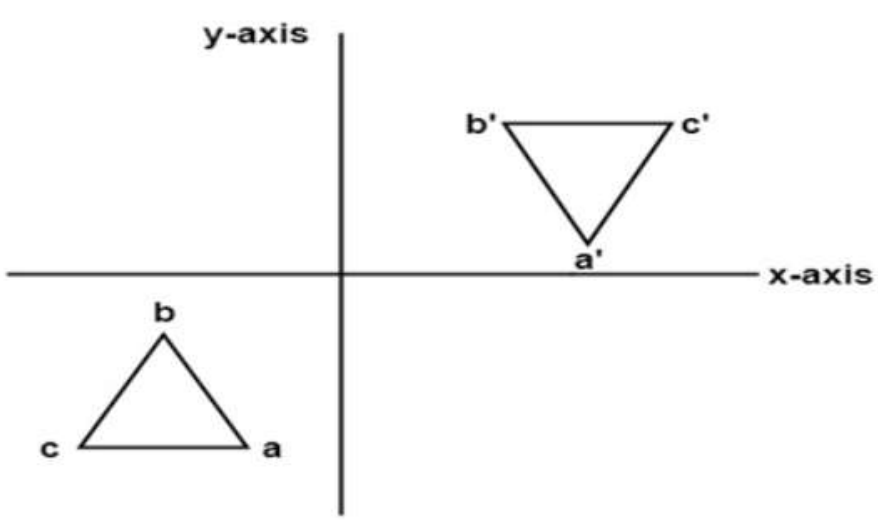


Here the values of x will be reversed, whereas the value of y will remain the same. The object will lie another side of the y-axis.

Reflection

3. Reflection about an axis perpendicular to xy plane and passing through origin: In the matrix of this transformation is given below

$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

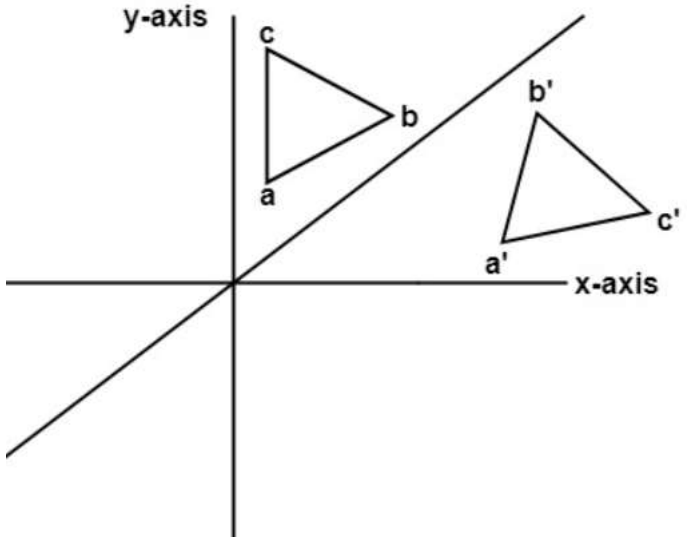


In this value of x and y both will be reversed. This is also called as half revolution about the origin

Reflection

4. Reflection about line y=x: The object may be reflected about line y = x with the help of following transformation matrix

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



First of all, the object is rotated at 45°. The direction of rotation is clockwise. After it reflection is done concerning x-axis. The last step is the rotation of y=x back to its original position that is counterclockwise at 45°.

Z-Buffer Algorithm

- When viewing a picture containing non transparent objects and surfaces, it is not possible to see those objects from view which are behind from the objects closer to eye. To get the realistic screen image, removal of these hidden surfaces is must. The identification and removal of these surfaces is called as the **Hidden-surface problem.**
- Z-buffer, which is also known as the Depth-buffer method is one of the commonly used method for hidden surface detection. It is an **Image space method**. Image space methods are based on the pixel to be drawn on 2D. For these methods, the running time complexity is the number of pixels times number of objects. And the space complexity is two times the number of pixels because two arrays of pixels are required, one for frame buffer and the other for the depth buffer.

Algorithm

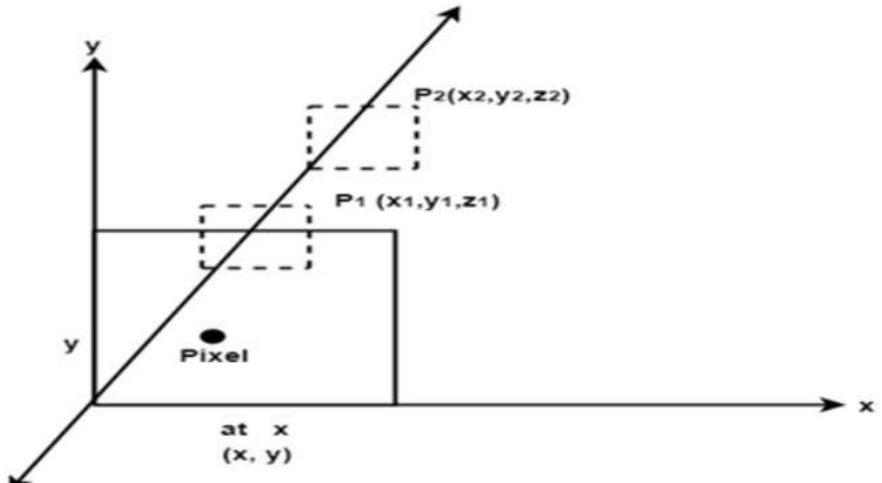
- For all pixels on the screen, set depth [x, y] to 1.0 and intensity [x, y] to a background value.
- For each polygon in the scene, find all pixels (x, y) that lie within the boundaries of a polygon when projected onto the screen. For each of these pixels:

  (a) Calculate the depth z of the polygon at (x, y)

  (b) If z < depth [x, y], this polygon is closer to the observer than others already recorded for this pixel. In this case, set depth [x, y] to z and intensity [x, y] to a value corresponding to polygon's shading. If instead z > depth [x, y], the polygon already recorded at (x, y) lies closer to the observer than does this new polygon, and no action is taken.
- After all, polygons have been processed; the intensity array will contain the solution.
- The depth buffer algorithm illustrates several features common to all hidden surface algorithms.

Algorithm

- First, it requires a representation of all opaque surface in scene polygon in this case.
- These polygons may be faces of polyhedral recorded in the model of scene or may simply represent thin opaque 'sheets' in the scene.
- The 2nd important feature of the algorithm is its use of a screen coordinate system. Before step 1, all polygons in the scene are transformed into a screen coordinate system using matrix multiplication.



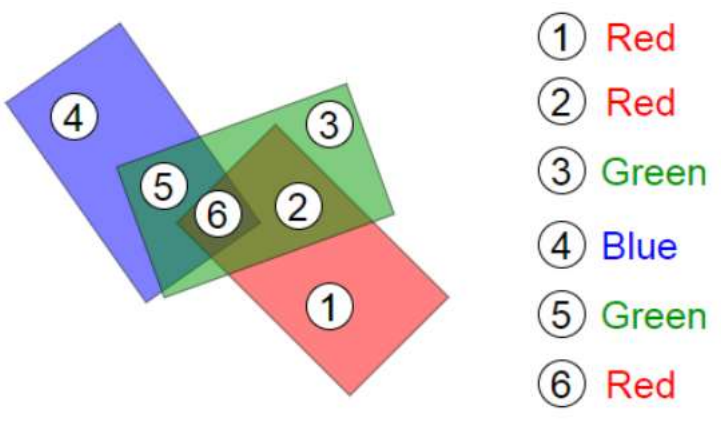
Points to remember

1) Z buffer method does not require pre-sorting of polygons.
2) This method can be executed quickly even with many polygons.
3) This can be implemented in hardware to overcome the speed problem.
4) No object to object comparison is required.
5) This method can be applied to non-polygonal objects.
6) Hardware implementation of this algorithm are available in some graphics workstations.
7) The method is simple to use and does not require additional data structure.
8) The z-value of a polygon can be calculated incrementally.
9) Cannot be applied on transparent surfaces i.e, it only deals with opaque surfaces.

Points to remember

10) If only a few objects in the scene are to be rendered, then this method is less attractive because of additional buffer and the overhead involved in updating the buffer.

11) Wastage of time may occur because of drawing of hidden objects.



Shading

- Shading is referred to as the implementation of the illumination model at the pixel points or polygon surfaces of the graphics objects.
- Shading model is used to compute the intensities and colors to display the surface. The shading model has two primary ingredients: properties of the surface and properties of the illumination falling on it. The principal surface property is its reflectance, which determines how much of the incident light is reflected. If a surface has different reflectance for the light of different wavelengths, it will appear to be colored.
- An object illumination is also significant in computing intensity. The scene may have to save illumination that is uniform from all direction, called diffuse illumination.

Shading

- Flat Shading
- each entire polygon is drawn with the same colour
- need to know one normal for the entire polygon
- lighting equation used once per polygon
- Given a single normal to the plane the lighting equations and the material properties are used to generate a single colour. The polygon is filled with that colour.

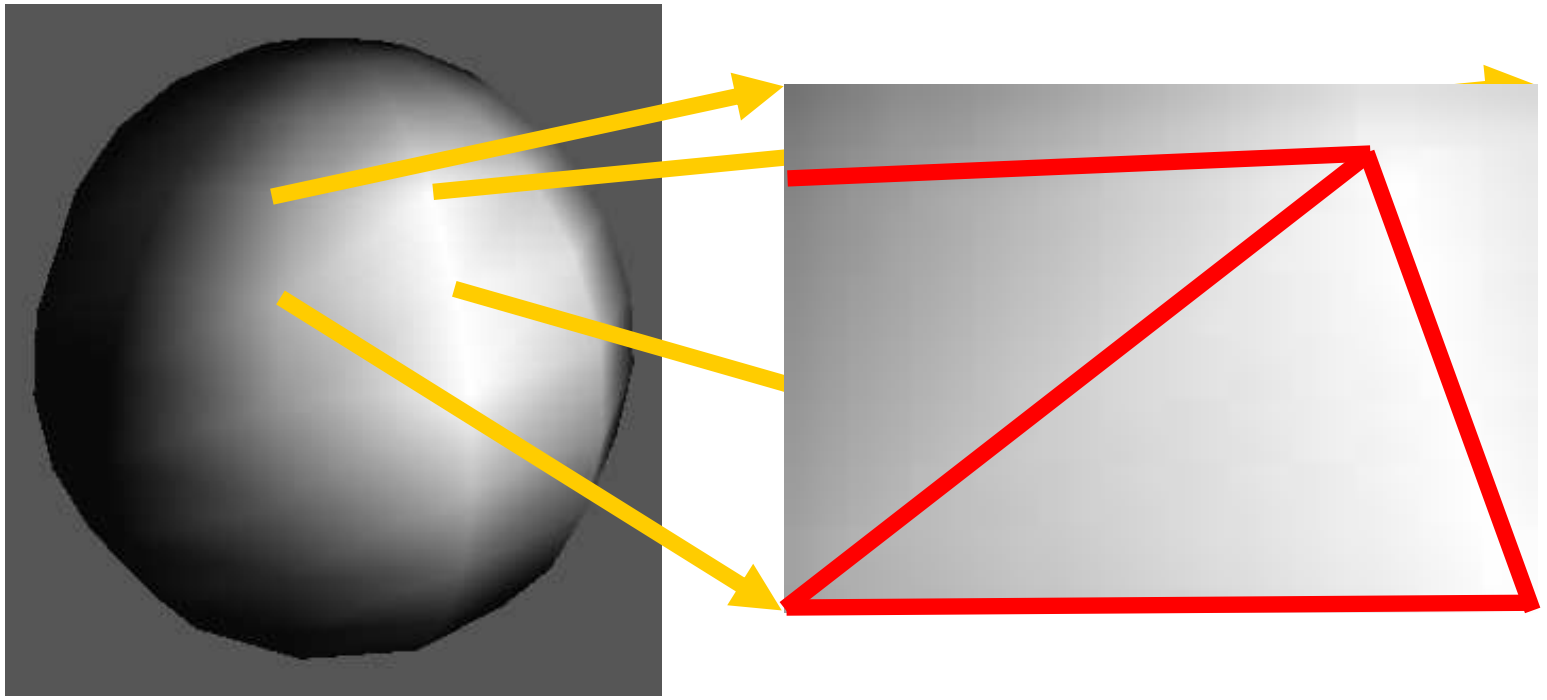
Constant Shading

- Constant Intensity or Flat Shading
- One color for the entire triangle
- Good for some objects
- What happens if triangles are small?
- Sudden intensity changes at borders



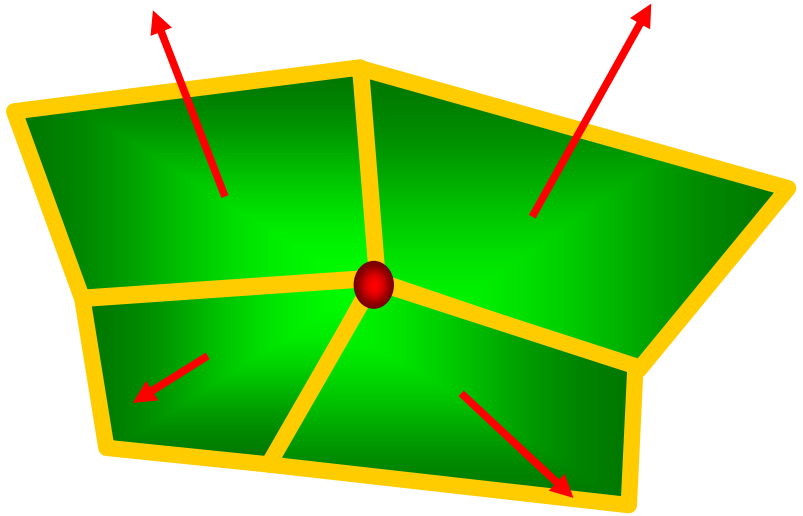
Gouraud Shading

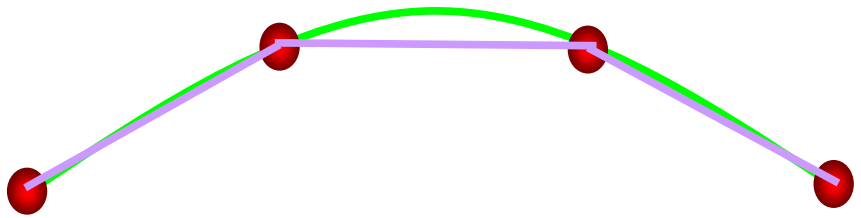
- Intensity Interpolation Shading
- Calculate lighting at the vertices. Then interpolate the colors as you scan convert



Gouraud Shading

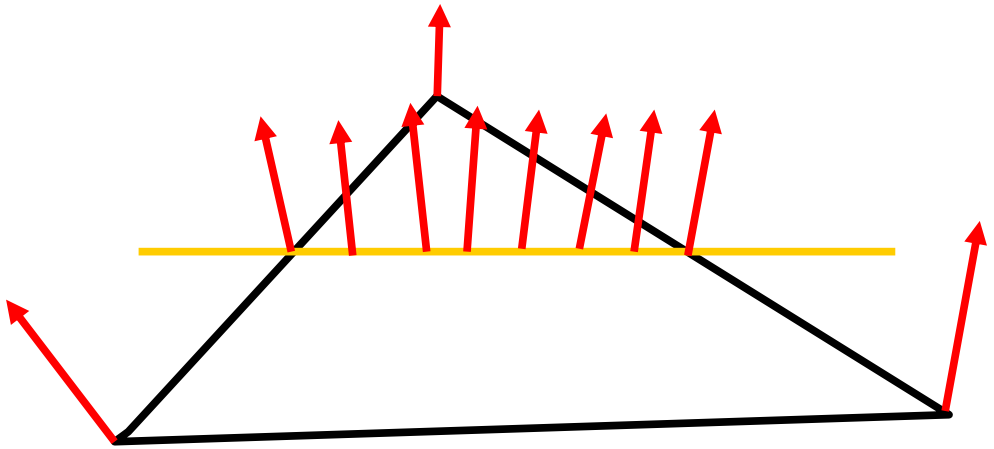
- Relatively fast, only do three calculations
- No sudden intensity changes





Phong Shading

- Interpolate the normal, since that is the information that represents the "curvature"
- Linearly interpolate the vertex normals. For ***each*** pixel, as you scan convert, *calculate* the lighting *per pixel*.
- True "per pixel" lighting
- Not done by most hardware/libraries/etc



Shading Techniques

- Constant Shading
  - Calculate one lighting calculation (pick a vertex) per triangle
  - Color the *entire* triangle the same color
- Gouraud Shading
  - Calculate three lighting calculations (the vertices) per triangle
  - Linearly interpolate the colors as you scan convert
- Phong Shading
  - While you scan convert, linearly interpolate the normals.
  - With the interpolated normal at each pixel, calculate the lighting at each pixel

Difference Between Gouraud Shading and Phong Shading

Sno	Gouraud Shading	Phong Shading
1.	Gouraud shading, named after Henri Gouraud.	Phong Shading model named after Bui Tuong Phong.
2.	Gouraud first published the technique in 1971.	Phong Shading published the technique in 1973.
3.	Computes illumination at border vertics and interpolates.	Illumination at every point of polygon surface.
4.	Interpolates colors along edges and scanline.	Interpolates normals instead of colors
5.	Not So expensive.	More Expensive than Gouraud Shading
6.	Requires Moderate processing and time.	Required Complex processing and it is Slower. Its Produce Good Quality.
7.	Lighting Equation used at each Vertex.	Lighting Equation used at each pixel.
8.	Gourard shading is in between the two: like Phong shading, each polygon has one normal vector per vertex, but instead of interpolating the vectors, the color of each vertex is computed and then interpolated across the surface of the polygon.	Each rendered polygon has one normal vector per vertex; shading is performed by interpolating the vectors across the surface and computing the color for each point of interest.
9.	Smooth surfaces.	Smooth and shining surfaces.

Difference Between Gouraud Shading And Phong Shading

